

COMP8051 - Operating System Engineering

Completion Date: 1st March 2024

Value: 15 marks

On completion please zip up your files and upload to Canvas.

On your laptop you may need to install qemu-system-i386 e.g. on Ubuntu:

```
sudo apt-get install qemu-system-i386
```

Clone the xv6 operating system from here using git.

```
git clone git://github.com/mit-pdos/xv6-public.git
```

run **make** to build xv6

and

make qemu

to run xv6

Execute some shell commands to get familiar with the user part of the OS. The shell commands are separate programs e.g. ls.c, cat.c.

To get a feel for how programs look in xv6, and how various APIs should be called, you can look at the source code for other utilities: echo.c, cat.c, wc.c, ls.c.

Hints for the questions below:

In places where something asks for a file descriptor, you can use either an actual file descriptor (i.e., the return value of the open function), or one of the standard I/O descriptors: 0 is "standard input", 1 is "standard output", and 2 is "standard error". Writing to either 1 or 2 will result in something being printed to the screen.

The standard header files used by xv6 programs are "types.h" (to define some standard data types) and "user.h" (to declare some common functions). You can look at these files to see what code they contain and what functions they defined.

Beware that programs don't have access to the typical stdio library that you expect to find on most systems. You'll have many of the functions you expect but some of the behavior might be different. For example, *printf* accepts an initial parameter that is the output stream: 1 represents the standard output and 2 represents the standard error stream. There is no FILE* type and no *fopen*, *fclose*, *fgets*, etc. calls. Look through **usertests.c** for examples on how all of the system calls provided with xv6 are used.

Question 1: hello world in xv6

Write a program for xv6 that, prints "Hello world" to the xv6 console. This can be broken up into a few steps:

1. Create the file `hello.c` in the `xv6` directory
2. Edit the file `Makefile`, find the section `UPROGS` (which contains a list of programs to be built), and add a line to tell it to build your `hello.c` code. When you're done that portion of the `Makefile` should look like:

```
UPROGS=\
_cat\
_echo\
_forktest\
_grep\
_init\
_kill\
_ln\
_ls\
_mkdir\
_rm\
_sh\
_stressfs\
_usertests\
_wc\
_zombie\
_hello\
```

3. Run **make** to build `xv6`, including your new program
4. Run **make qemu** to launch `xv6`, and then type `hello` in the QEMU window. You should see "Hello world" be printed out.

(0 marks)

Question 2: head command

Write a program that prints the first 10 lines of its input for the `xv6` operating system. If a filename is provided on the command line (i.e., **head** FILE) then **head** should open it, read and print the first 10 lines, and then close it. If no filename is provided, **head** should read from standard input.

See how the program `cat.c` works e.g

```
cat README
```

grep the README | head

should show ten lines each line has the word the in it.

Many aspects of this are similar to the `wc` program: both can read from standard input if no arguments are passed or read from a file if one is given on the command line. Reading its code will help you if you get stuck.

(2 marks)

Question 3: Trace

Add a new system call called *trace* to *xv6*. Its syntax is

```
int trace(int)
```

When called with a non-zero parameter, e.g., *trace(1)*, system call tracing is turned on for that process. Each system call from that process will be printed to the console in a user-friendly format showing:

- the process ID
- the process name
- the system call number
- the system call name

Any other processes will not have their system calls printed unless they also call *trace(1)*.

Calling *trace(0)* turns tracing off for that process. System calls will no longer be printed to the console.

In all cases, the *trace* system call also returns the total number of system calls that the process has made. Hence, you can write code such as:

```
printf("total system calls so far = %d\n", trace(0));
```

The system call counting for each system call on a per-process basis. You will need to keep track of this in the process control block, the **proc** structure.

Write a test program (*try.c*) to test your *trace* system call.

See Appendix 1 for more details.

(4 marks)

Question 4: system calls and interrupts

Read the *xv6* book chapter 3, *vector.S*, *trapasm.S*, *trap.c*, and *syscall.c* and briefly describe how *xv6* manages interrupts and system calls.

(4 marks)

Question 5: find

Write a simple version of the *find* program for *xv6*: find all the files in a directory tree whose name matches a string.

Some hints:

- Look at *ls.c* to see how to read directories.
- Don't try to find any files in the "." and ".." directories.

- Changes to the file system persist across runs of QEMU; to get a clean file system run `make qemu`.

Your solution is correct if produces the following output (when the file system contains a file `a/b`):

```
$ make qemu
...
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ find . b
./b
./a/b
$
```

(2 marks)

Question 6: Linux tracing

Give an overview of how a superuser attacker would use Linux tools such as `strace` and `bcc` to dump all the contents of the HTTP requests to/from a web server by inspecting read, write system calls. You are to describe how the attacker would dump the HTTP protocol headers and the request/response contents.

Scenario

On linux all networking and file system interactions with the kernel use system calls such as *read* and *write* to interact with the networking stack and/or the file system.

The attacker could for example use a command like

```
ps -ale | grep node
```

to find the process id of the web server (assuming a nodejs based web server). The attacker could for example dump all system calls from a process with id 4563 with the command

```
sudo strace -p 4563
```

But this command only dumps the first 20 bytes of the read/write request. The attacker would therefore need to further refine his/her methodology to uncover the full request/response message contents.

Method

See [here](https://nodejs.org/en/learn/getting-started/introduction-to-nodejs) for `server.js` which is a nodejs web server

<https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>

Node can be installed on Ubuntu using the command:

```
sudo apt install nodejs
```

you can run the web server using the commands

```
node server.js
```

NOTE

The USDT (User Statically-Defined Tracing) probes that are used in this article

<https://www.brendangregg.com/blog/2016-10-12/linux-bcc-nodejs-usdt.html>

no longer exist as these probes have been removed from the nodejs runtime.

See here for a discussion

<https://opeonikute.dev/posts/observing-node-with-ebpf>

References

<https://github.com/iovisor/bcc/blob/master/docs/tutorial.md>

<https://opeonikute.dev/posts/observing-node-with-ebpf>

<https://www.youtube.com/watch?v=nxQ-V9yrUNU>

<https://blogs.oracle.com/linux/post/intro-to-bcc-2>

(3 marks)

Appendix 1

How to add a new system call to XV6

You need to touch several files to add a system call in xv6. Look at the implementation of existing system calls for guidance on how to add a new one.

The files that you need to edit to add a new system call include:

user.h

This contains the user-side function prototypes of system calls as well as utility library functions (stat, strcpy, printf, etc.).

syscall.h

This file contains symbolic definitions of system call numbers. You need to define a unique number for your system call. Be sure that the numbers are consecutive.

That is, there are no missing number in the sequence. These numbers are indices into a table of pointers defined in `syscall.c` (see next item).

`syscall.c`

This file contains entry code for system call processing. The `syscall(void)` function is the entry function for all system calls. Each system call is identified by a unique integer, which is placed in the processor's `eax` register. The `syscall` function checks the integer to ensure that it is in the appropriate range and then calls the corresponding function that implements that call by making an indirect function call to a function in the `syscalls[]` table. You need to ensure that the kernel function that implements your system call is in the proper sequence in the `syscalls` array.

`usys.S`

This file contains macros for the assembler code for each system call. This is user code (it will be part of a user-level program) that is used to make a system call. The macro simply places the system call number into the `eax` register and then invokes the system call. You need to add a macro entry for your system call here.

`sysproc.c`

This is a collection of process-related system calls. The functions in this file are called from `syscall`. You can add your new function to this file. Per-process state is stored in a `proc` structure: `struct proc` in `proc.h`. You'll need to extend that structure to keep track of the process related metrics. You'll also need to find where the `proc` structure is allocated so that you can ensure that the elements are initialized appropriately.

When you implement your trace call, you'll need to retrieve the incoming parameter. The file `sysproc.c` defines a few helper functions to do this. The functions `argint`, `argptr`, and `argstr` retrieve the n th system call argument, as either an integer, pointer, or a string. `argint` uses the `esp` register to locate the argument: `esp` points at the return address for the system call stub.

Implementation steps

1. Write a test program.

Add the test program to the Makefile so that it will be compiled and built whenever you run `make`.

In the Makefile, add your program (e.g., `try.c`) to the list of user commands in the `UPROGS=` section. That should be all you need to do to that file.

Beware that programs don't have access to the typical `stdio` library that you expect to find on most systems. You'll have many of the functions you expect but some of the behavior might be different. For example, `printf` accepts an initial parameter that is the output stream: `1` represents the standard output and

2 represents the standard error stream. There is no FILE* type and no fopen, fclose, fgets, etc. calls. Look through usertests.c for examples on how all of the system calls provided with xv6 are used.

2. Add system call tracing to the kernel.

Print a message identifying every system call that is requested by any process as well as the process ID and process name. You do not need to print the arguments to the system calls. When you run any program, including the shell, you will see output similar to this:

```
...
pid: 2 [sh] syscall(5=read)
pid: 2 [sh] syscall(5=read)
pid: 2 [sh] syscall(1=fork)
pid: 2 [sh] syscall(3=wait)
pid: 3 [sh] syscall(12=sbrk)
pid: 3 [sh] syscall(7=exec)
pid: 3 [try] syscall(20=mkdir)
pid: 3 [try] syscall(15=open)
pid: 3 [try] syscall(16=write)
pid: 3 [try] syscall(21=close)
pid: 3 [try] syscall(2=exit)
pid: 2 [sh] syscall(16=write)
pid: 2 [sh] syscall(16=write)
...
```

Use the cprintf function in the kernel, which prints using direct access to the vga controller. It works just like the normal Linux printf function. For example:

```
cpprintf("hello, I'm number %d\n", num);
```

3. Restrict this output to a single process.

Create a new system call called trace(int). This turns console-based system call logging on and off for only the calling process. Extend the proc structure (the process control block) in proc.h to keep track of whether tracing for the process is on or off. Be sure that the elements are cleared (initialized) whenever a new process is created. In implementing your system call, you'll need to access the

single parameter passed by trace. Use the helper functions defined in `syscall.c` (`argint`, `argptr`, and `argstr`). Take a look at how other system calls are implemented in `xv6`. For example, `getpid` is a simple system call that takes no arguments and returns the current process' process ID; `kill` and `sleep` are examples of system calls that take a single integer parameter.

4. Add system call counting.

Be sure to count calls on a per-process basis. You will need to keep track of this in the process control block, the `proc`